

GraphRAG+知识图谱可视化--凡人修仙传

Graph RAG (Graph Retrieval-Augmented Generation) 是传统 RAG (Retrieval-Augmented Generation) 的一种扩展形式，它通过引入知识图谱 (Knowledge Graph) 来增强信息检索和生成的质量。下面我们从核心逻辑、区别以及一个通俗案例来详细解释。

Graph RAG 的核心逻辑

Graph RAG 的核心思想是：

利用知识图谱中的结构化关系信息，辅助模型更精准地理解查询意图，并在海量数据中找到最相关的信息进行生成。

它的关键技术包括：

1. 构建或使用已有知识图谱：将实体与实体之间的关系以图的形式表示。
2. 图检索 (Graph-based Retrieval)：根据用户问题，在知识图谱中查找相关的实体和关系。
3. 图增强生成 (Graph-enhanced Generation)：结合图中检索到的路径或子图信息，生成更准确、连贯的回答。

这种方式不仅依赖文档文本，还利用了实体之间的语义关联，从而提升了回答的深度和准确性。

环境配置

One Api安装和部署

OneAPI是什么

官方介绍：是OpenAI接口的管理、分发系统, 支持 Azure、Anthropic Claude、Google PaLM 2 & Gemini、智谱 ChatGLM、百度文心一言、讯飞星火认知、阿里通义千问、360 智脑以及腾讯混元



项目地址: <https://github.com/songquanpeng/one-api>

```
D:\rag_project\graph_test\oth: x + v
[SYS] 2025/07/07 - 15:09:52 | One API v0.6.8 started
[SYS] 2025/07/07 - 15:09:52 | SQL_DSN not set, using SQLite as database
[SYS] 2025/07/07 - 15:09:52 | database migration started
[SYS] 2025/07/07 - 15:09:52 | database migrated

2025/07/07 15:09:52 D:/a/one-api/one-api/model/main.go:27 record not found
[0.000ms] [rows:0] SELECT * FROM `users` ORDER BY `users`.`id` LIMIT 1
[SYS] 2025/07/07 - 15:09:52 | no user exists, creating a root user for you: username is root, password is 123456
[SYS] 2025/07/07 - 15:09:52 | REDIS_CONN_STRING not set, Redis is not enabled
[SYS] 2025/07/07 - 15:09:52 | using theme default
[SYS] 2025/07/07 - 15:09:52 | initializing token encoders
[SYS] 2025/07/07 - 15:09:53 | token encoders initialized
[SYS] 2025/07/07 - 15:09:53 | server started on http://localhost:3000
```

创建渠道和令牌

构建 GraphRAG

2.1 下载源码

一键三连, 评论区领取

2.2 构建项目

使用 PyCharm 创建一个新的 Python 项目，并为其配置虚拟环境。

- 项目名称： `GraphragTest`

2.3 将相关代码拷贝到项目工程中

直接将下载的文件夹中的所有内容复制到新建项目的目录中。

2.4 安装项目依赖

在终端运行以下命令安装所需的依赖包：

```
pip install -r requirements.txt
```

⚠ 注意：每个软件包都指定了本次测试所用的固定版本号。本视频使用的 `graphrag` 版本为 0.3.0（最新）。

2.5 创建 GraphRAG 所需文件夹

在当前项目下创建一个自定义文件夹 `ragtest`，后续操作均在此目录下进行。

```
mkdir ragtest
```

```
cd ragtest
```

```
mkdir input
```

```
mkdir inputs
```

```
mkdir cache
```

2.6 准备测试文档

我们以《凡人修仙传》白话文前4章作为测试文本：

⚠ 将 `other/text/` 目录下的 `1-4.txt` 文件复制到 `ragtest/input` 文件夹中。

2.7 初始化 GraphRAG

执行以下命令初始化 GraphRAG 索引结构：

```
python -m graphrag.index --init --root ./
```

2.8 设置参数

编辑 `.env` 和 `settings.yaml` 文件：

💡 提示：可参考 `other/temp` 文件夹下的 `.env` 和 `settings.yaml` 示例文件，直接复制即可适配阿里通义千问大模型。

2.9 优化提示词

运行以下命令对提示词进行调优（适合中文）：

```
python -m graphrag.prompt_tune --config ./settings.yaml --root ./ --no-entity-types --language Chinese --output ./prompts
```

2.10 构建索引

最后构建完整的知识图谱索引：

```
python -m graphrag.index --root ./
```

3、测试 GraphRAG

请将 `utils` 文件夹复制到 `ragtest` 根目录下以便进行测试。

3.1 运行 main.py 脚本

⚠ 注意事项：

- 修改 `main.py` 中的输入路径为你的实际路径，例如：
 - `INPUT_DIR =`
 - `"/your/project/path/GraphragTest/ragtest/inputs/artifacts"`
配置好大模型连接信息。
 - 指定向量数据库集合名（如 `entity_description_embeddings`），根据实际情况修改：
 - `description_embedding_store =`
`LanceDBVectorStore(collection_name="entity_description_embeddings")`
-

3.2 运行 apiTest.py 测试接口

在另一个终端窗口中运行如下命令启动 API 测试：

```
python apiTest.py
```

确保 `main.py` 已成功运行后再执行此步骤。

3.3 使用 Neo4j 可视化知识图谱

访问 Neo4j 在线平台并注册账号，创建实例后打开查询界面：

 [Neo4j 在线平台](#)

⚠ 注意事项：

- 修改 `neo4jTest.py` 中的路径为你的项目路径：
 - `GRAPHRAG_FOLDER="/your/project/path/GraphragTest/ragtest/inputs/artifacts"`
 - 配置 Neo4j 的数据库连接信息（用户名、密码、URI 等）。
-

3.4 使用 3D 图可视化知识图谱

运行以下脚本以生成交互式 3D 图谱：

⚠ 修改 `graphrag3dknowledge.py` 中的路径为你的实际路径：

```
directory = '/your/project/path/GraphragTest/ragtest/inputs/artifacts'
```

🎉 至此，GraphRAG 的构建与测试流程已完成！你可以通过 API 查询、Neo4j 或 3D 图来探索知识图谱的强大功能了！